Prepared by Sai Veda

NL-to-SQL Lakehouse

Author	Document date	Validation
Sai Veda	March 11, 2024	36 automated tests

Abstract

This brief captures a natural-language analytics system built on a real lakehouse pattern: a large partitioned warehouse, a semantic layer, and a deterministic text-to-SQL compiler with safety checks.

Project Background

Business users ask in plain language, but trusted reporting still depends on correct joins, guarded metrics, and a stable schema contract. The goal was to bridge those worlds without hiding behind an opaque model call.

Headline Result

50M-row lakehouse; 100% NL->SQL execution accuracy

Scope Overview

Fact and dimension generation into partitioned Parquet. DuckDB execution layer over the lakehouse. Entity linking, metric mapping, and validated SQL generation. Benchmarks on full scans, joins, grouped queries, and distinct counts.

Project Context

A DuckDB lakehouse over a partitioned-Parquet star schema, with a semantic layer and a deterministic, offline text-to-SQL engine. Ask a business question in English - "top 5 product categories by revenue", "which region has the lowest revenue?", "monthly revenue in 2023" - and the system links schema entities, compiles validated SQL through the semantic layer, and executes it out-of-core against 50M fact rows.

Executive Summary

Background, objectives, scope, and project impact.

Objectives

- Stand up a large star-schema lakehouse that remains queryable out of core.
- Translate common business questions into safe, deterministic SQL.
- Anchor generated SQL to a semantic layer instead of raw table guesses.

Scope

- Fact and dimension generation into partitioned Parquet.
- DuckDB execution layer over the lakehouse.
- Entity linking, metric mapping, and validated SQL generation.
- Benchmarks on full scans, joins, grouped queries, and distinct counts.

Project Background

Business users ask in plain language, but trusted reporting still depends on correct joins, guarded metrics, and a stable schema contract. The goal was to bridge those worlds without hiding behind an opaque model call.

Executive Summary

This brief captures a natural-language analytics system built on a real lakehouse pattern: a large partitioned warehouse, a semantic layer, and a deterministic text-to-SQL compiler with safety checks.

Impact

The project demonstrates that natural-language analytics becomes much more credible when it is backed by a semantic layer and SQL validation. The result is a system that answers business questions in English while still producing auditable SQL.

50M-row lakehouse; 100% NL->SQL execution accuracy

Solution Architecture

The semantic layer and SQL guardrails are the load-bearing parts that keep natural-language analytics trustworthy.

Core system flow

Design Note 1

The semantic layer is what makes NL-to-SQL trustworthy rather than merely executable.

Design Note 2

DuckDB was chosen because it keeps the warehouse story local, fast, and transparent.

Design Note 3

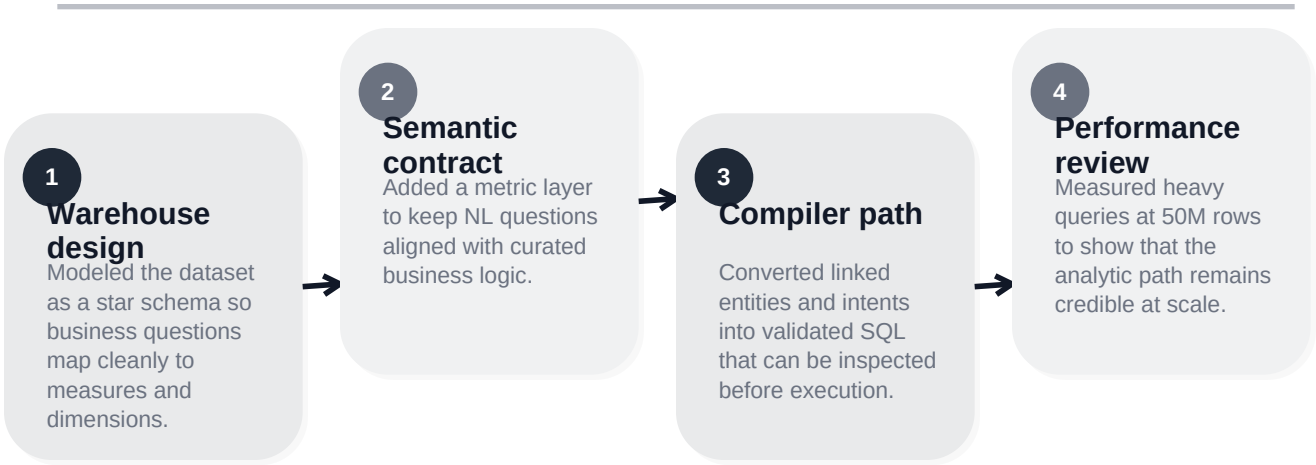
Validation is a product feature: unsafe or schema-breaking queries should fail early and clearly.

Implementation Process

Delivery phases, engineering approach, and quality checkpoints.

Delivery Narrative

A DuckDB lakehouse over a partitioned-Parquet star schema, with a semantic layer and a deterministic, offline text-to-SQL engine. Ask a business question in English - "top 5 product categories by revenue", "which region has the lowest revenue?", "monthly revenue in 2023" - and the system links schema entities, compiles validated SQL through the semantic layer, and executes it out-of-core against 50M fact rows.



Quality Checkpoints

- 36 automated tests validate core behavior and edge cases.
- Benchmark files and screenshots are generated from committed project artifacts.
- The run path stays deterministic so numbers can be reproduced and reviewed directly.

Engineering Principles

- The semantic layer is what makes NL-to-SQL trustworthy rather than merely executable.
- DuckDB was chosen because it keeps the warehouse story local, fast, and transparent.
- Validation is a product feature: unsafe or schema-breaking queries should fail early and clearly.

Evidence And Results

Test evidence, benchmark interpretation, and screenshot review.

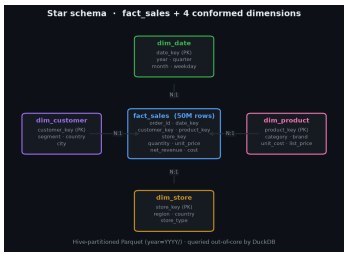
36 passing tests | **March 11, 2024** | **50M-row lakehouse; 100% NL->SQL execution accuracy**

Benchmark Evidence

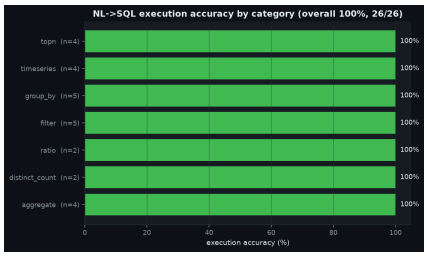
This repository emphasizes visual benchmark artifacts and automated validation outputs. Where tables are not present, the screenshots and test fixtures remain the primary review surface.

Result Interpretation

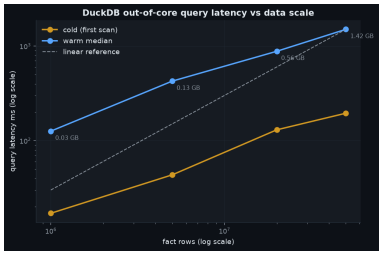
- Execution accuracy matters because every natural-language answer is backed by runnable SQL, not narrative text alone.
- The lakehouse benchmarks show that large-table scans and joins remain practical on one machine.
- The screenshots reinforce that trust comes from visible SQL and visible schema



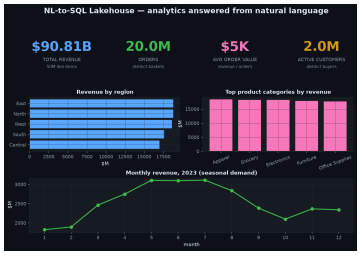
Star schema (data model)



NL->SQL execution accuracy by category



Query latency vs data scale (log-log)



Analytics answered from natural language

Risks And Next Steps

Operational considerations, mitigation choices, and forward path.

Risk Register

Schema ambiguity	Handled through explicit entity linking and a semantic metric layer.
Unsafe SQL	Managed with validation gates before query execution is allowed.
Metric drift	Reduced by centralizing business definitions instead of encoding them ad hoc in prompts.

Next Steps

- Support more complex cohort, funnel, and time-series comparison queries.
- Add approval workflows for generated SQL in regulated or finance-facing contexts.
- Connect the semantic layer to upstream catalog metadata and freshness indicators.

Closing Note

The project brief is intentionally written as delivered work rather than a transfer note. It documents the business problem, the engineering choices, the measured evidence, and the remaining operational path in a format suitable for portfolio review.

Sai Veda

Project brief date: March 11, 2024

50M-row lakehouse; 100% NL->SQL execution accuracy